> is on the future roadmap. I mention that now in case it's a deal-breaker. It's more or less the position of the Cypress lead developer that running the same tests in multiple browsers is not tremendously valuable, but that said, Safari has some quirks and niches where many teams would likely want it as their primary test browser.

Cypress also has a command-line version of the app:

```
$ rails cypress:run
```

This version will run all the tests without the UI and is suitable for use in a continuous integration server. By default, the command-line version of the tool will take a snapshot of failing steps and put them in `tmp/cypress/snapshots`. It will also take a video of all steps and place it in `tmp/cypress/videos`. The videos will automatically be deleted before the next run.

One more nice-to-have before we start writing our own tests is integration with the ESLint system so that our Cypress tests don't cause lint errors. Add the following NPM module:

```
$ yarn add eslint-plugin-cypress --dev
```

In the `extends` key in the `eslintrc.js` file, add `"plugin:cypress/recommended"` to that list. Now, ESLint will be aware of Cypress testing commands (you may need to restart your editor for it to take effect).

An important thing to note is that given our setup here, just calling the Cypress commands does not include our watch commands for JavaScript and CSS builds, meaning that any JavaScript or CSS changes will not automatically be rebuilt. To incorporate JS or CSS changes, you need to either keep your `build/dev` command running, or you need to create a separate `Procfile` for testing that includes the Cypress command and the watch commands in our development Procfile. Which one works for you is going to depend on how you structure your setup—you don't want both Cypress and the dev build running watches, for example.